

ASSESSMENT AND INTERNAL VERIFICATION FRONT SHEET (Individual Criteria)

(Note: This version is to be used for an assignment brief issued to students via Classter)

Course Title	BSc. Digital Games Development				Lecturer Name & Surname	David Deguara		
Unit Number & Title		ITDGM-506-2303 Procedural Content Generation						
Assignment Number, Title / Type		Home Assignment – Developing procedurally oriented games						
Date Set		20/03/2026		Deadline Date	22/05/2026			
Student Name				ID Number		Class / Group		

Assessment Criteria	Maximum Mark
<i>R&U3: Describe how the graphics pipeline renders graphics for digital games</i>	5
<i>R&U6: Describe how noise, voronoi tessellation, midpoint displacement, splat maps and L-systems can be used to generate terrains</i>	5
<i>A&A2: Produce a code that generates shapes</i>	7
<i>A&A3: Produce a code for generating trees and vegetation</i>	7
<i>A&A4: Produce a code for generating natural elements</i>	7
<i>E&C1: Generate meshes and shapes through code in games</i>	10
<i>E&C2: Generate random terrains</i>	10
<i>E&C3: Generate random levels in games</i>	10
Total Mark	61

Notes to Students:

- This assignment brief has been approved and released by the Internal Verifier through Classter.
- Assessment marks and feedback by the lecturer will be available online via Classter ([Http://mcast.classter.com](http://mcast.classter.com)) following release by the Internal Verifier
- Students submitting their assignment on Moodle/Turnitin will be requested to confirm online the following statements:
 - Student's declaration prior to handing-in of assignment**
 - ❖ I certify that the work submitted for this assignment is my own and that I have read and understood the respective Plagiarism Policy
 - Student's declaration on assessment special arrangements**
 - ❖ I certify that adequate support was given to me during the assignment through the Institute and/or the Inclusive Education Unit.
 - ❖ I declare that I refused the special support offered by the Institute.

Procedural Content Generation

Home Assignment



Engine: Unity 6 (URP 2D)

Language: C#

Total Marks: 61

Academic Year 2025/26

1. Introduction

In this assignment you will design and develop a top-down 2D dungeon crawler in Unity 6, where every element of the game world is generated procedurally at runtime. Rather than hand-crafting multiple levels, your game will feature a single, continuously regenerating dungeon whose difficulty adapts dynamically to how well the player is performing.

You will work with Unity's Tilemap system to build the dungeon, populating it with procedurally placed rooms, corridors, enemies, traps, vegetation and environmental hazards. The entire project should demonstrate your understanding of procedural content generation (PCG) algorithms and your ability to implement them in a real game engine.

Important: This is an individual assignment. You must work alone and submit your own original work. MCAST's academic integrity policy applies in full.

2. Assignment Overview

Your finished game must be a playable, top-down 2D dungeon crawler built entirely in Unity 6 using the Universal Render Pipeline (URP) for 2D. The player navigates through a procedurally generated dungeon, defeating enemies, avoiding traps and collecting items. There are no hand-made levels; instead, the dungeon regenerates each time and the game's difficulty scales dynamically based on player performance metrics such as time taken, kill count, death count, damage received and accuracy.

3. Assignment Tasks

Task 1 — AI-Generated Dungeon Tileset (Prerequisite)

This task does not carry a separate mark allocation. However, your tileset is a required component of the project and its quality will directly affect your marks in Tasks 2–5, where it is used to render the dungeon. A poor or missing tileset will limit what you can achieve in every subsequent task.

Before you write any code, you need a visual foundation for your dungeon. Using a generative AI image tool (Flux 2 Pro, Gemini, Claude, Midjourney or a similar service), generate a cohesive, top-down pixel-art dungeon tileset sprite sheet. This tileset will be imported into Unity and sliced for use with the Tilemap system.

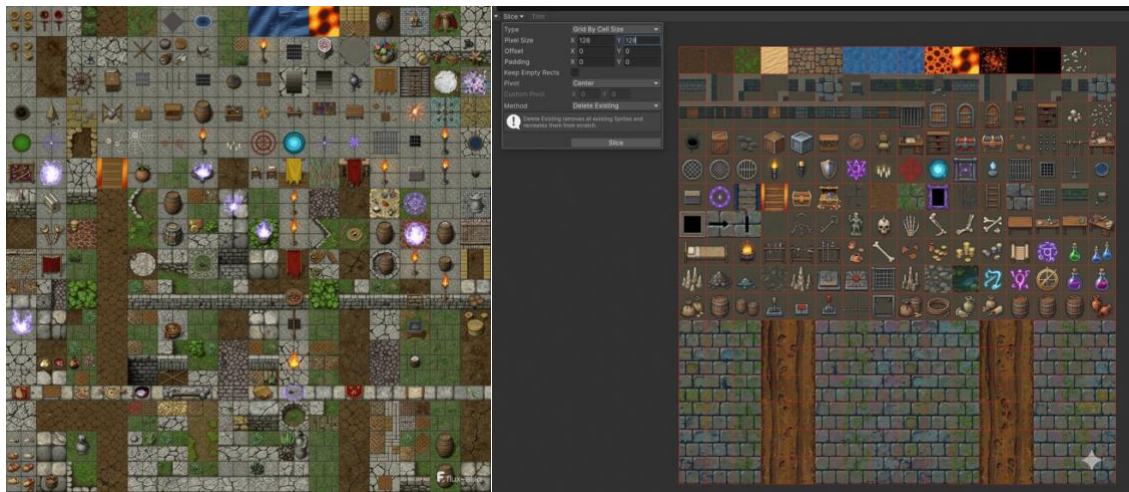
Be aware that generating a truly cohesive, grid-aligned tileset is not a trivial task. Generative AI tools frequently produce tiles that are misaligned, inconsistent in style or that bleed across tile boundaries. You should expect to iterate on your prompts many times, refine results in an image editor and test your tileset in Unity repeatedly before achieving a usable result. Patience and methodical improvement are essential here.

Use a prompt similar to the one below as a starting point and refine it until the output meets your needs:

Example Prompt

“Create a professional pixel-art dungeon tileset sprite sheet for a top-down 2D dungeon crawler intended for Unity Tilemap workflows. The output must be a single large image composed of a perfectly aligned grid where every tile is exactly 64×64 pixels, with no padding, no margins, no borders, no spacing and no overlapping elements, so that Unity can automatically slice the sheet into clean 64×64 tiles. All tiles must touch edge-to-edge and be arranged in a strict orthogonal grid. The perspective must be true top-down orthographic (90° overhead) — not isometric, not angled, not perspective — with consistent directional lighting and shadows that remain fully inside each tile and never bleed across tile boundaries. Use high-quality, professional pixel art with clean pixel clusters, crisp edges, limited colour palette and deliberate dithering, avoiding painterly or blurry textures. The visual style should depict a dark medieval fantasy dungeon with aged stone blocks, cracked tiles, moss growth, grime, chipped edges, damp stains, subtle colour variation and believable wear, ensuring consistent scale, shading and colour palette across the entire sheet. All terrain tiles must be fully tileable and seamlessly repeatable in all directions so they can be used for procedural dungeon generation and auto-tiling systems. Include a large, well-organised variety of dungeon tiles: base terrain tiles such as stone floor, cracked floor, mossy floor, dirt, gravel, decorative floor mosaics, puddled water, flowing water, lava, glowing magma cracks and deep pit/void tiles; modular wall pieces including horizontal walls, vertical walls, inner corners, outer corners, T-junctions, cross intersections, thick pillars, damaged walls, cracked walls, mossy walls and broken wall segments; door systems including wooden doors (open and closed), reinforced doors, locked doors, iron gates (open and closed), portcullis gates, barred doors and secret doors disguised as walls; vertical traversal tiles including stairs down, stairs up, spiral stair entrances, trapdoor ladders and ladder shafts; environmental obstacles such as rock piles, rubble heaps, cracked floor holes, broken columns, collapsed masonry debris and wooden barricades; interactive props including treasure chests (open and closed), barrels, crates, sack piles, wooden tables, bookshelves, weapon racks and storage clutter; trap tiles including spike traps, flame traps, arrow trap wall tiles, poison pools, rune traps and pressure plates; atmospheric decorations including skulls, scattered bones, spider webs, blood stains, wall torches, candle clusters, ritual circles, glowing magical portals, arcane sigils and crystal pedestals; and dungeon structures such as iron prison bars, dungeon windows with bars, hanging cages, wells, sacrificial altars, rune circles, pit edges and bridge tiles for crossing water or lava. Every object must fit completely inside its 64×64 tile without clipping, maintain consistent scale relative to other tiles and be designed so each tile works independently within a procedural tilemap system. The final sprite sheet must appear cohesive, readable at gameplay scale, visually consistent and immediately usable in a Unity tilemap pipeline without additional editing or slicing adjustments.”

You are free to adapt and improve this prompt, generate multiple sheets or combine AI output with manual pixel-art edits. Document your prompt iteration process in your video presentation.



The images above show tilemap examples using Flux and Google Gemini.

Important — characters, enemies and NPCs: The AI-generated tileset requirement applies to the dungeon environment tiles (floors, walls, doors, props, traps, decorations and so on). Your player character, enemy sprites and NPC sprites do **not** need to be AI-generated. You are free to create these by hand, source them from asset packs (with proper attribution) or use any other method you prefer. The focus of this task is on producing a cohesive, tileable environment tileset through generative AI.

Task 2 — Procedural Dungeon Layout Generation [20]

Criteria assessed: *R&U3, R&U6, E&C3*

This is the core of your project and carries the largest mark allocation. Implement a procedural dungeon generation algorithm that creates the layout of your dungeon at runtime using Unity's Tilemap system. Your generator must produce a playable dungeon with interconnected rooms and corridors each time it is run. You must demonstrate an understanding of the graphics pipeline that renders your tilemap to screen and apply recognised PCG terrain techniques within your generation logic.

Requirements:

Your dungeon generator must, at minimum, incorporate the following elements:

Room generation: Procedurally determine the number, size, shape and position of rooms. You may use algorithms such as Binary Space Partitioning (BSP), random room placement with overlap checking, cellular automata or a combination of methods.

Corridor generation: Connect rooms with corridors using techniques such as minimum spanning trees, A* pathfinding between room centres, flood-fill or L-shaped/random-walk corridors. Ensure all rooms are reachable.

Tilemap painting: Use your AI-generated tileset (Task 1) to paint floors, walls, corners, doors and other structural elements onto the Unity Tilemap. Implement proper wall auto-tiling so that wall tiles face the correct direction based on their neighbours.

Spawn points: Designate a player spawn location and an exit point (e.g. stairs, portal) at appropriate positions within the generated layout.

Seed-based generation: Support an optional seed value so that a specific dungeon layout can be reproduced for testing and debugging.

Task 3 — Procedural Terrain and Environmental Variation [10]

Criterion assessed: *E&C2*

With the structural layout in place, this task focuses on adding visual and gameplay variety to the dungeon's terrain. Rather than painting every room with the same stone floor, use procedural techniques to create environmental variation that makes each area of the dungeon feel distinct.

Requirements:

Noise-driven tile variation: Use Perlin noise, Simplex noise or similar functions to vary floor tiles across the dungeon. For example, sample a noise value at each floor tile's position and use thresholds to assign different tile types: dry stone, cracked stone, mossy stone, damp stone or shallow puddles. This creates organic-looking texture variation rather than uniform floors.

Environmental zones or biomes: Divide the dungeon into procedurally assigned zones, each with its own visual theme and tile palette. For instance, one cluster of rooms might have a flooded/waterlogged theme with puddle tiles and damp walls, whilst another might feature a volcanic theme with lava cracks and scorched stone. Zone assignment could be driven by distance from the spawn point, Voronoi regions or random clustering.

Procedural decoration placement: Scatter decorative tiles (rubble, bones, cobwebs, bloodstains, torch sconces, cracks) across the dungeon using probability-based rules. Decorations should respect context — torches belong on walls, rubble on floors near damaged walls and so on.

Terrain-based gameplay effects (recommended): Consider giving terrain types gameplay significance. Water tiles might slow the player, lava tiles might deal damage and mossy tiles might indicate hidden passages. This links your procedural terrain directly to the gameplay experience.

Task 4 — Procedural Object, Trap and Enemy Spawning [17]

Criteria assessed: A&A2, E&C1

This task covers the procedural generation of all interactive game objects — enemies, traps, loot and dungeon furniture. These objects give the dungeon its gameplay substance and are where your procedural systems will have the most visible impact on the player experience.

Requirements:

Procedural enemy spawning: Populate rooms with enemies whose type, quantity and placement are determined procedurally. Use rules such as room size, distance from spawn and the current difficulty level (see Task 6) to govern spawning. Implement at least three distinct enemy types with different behaviours (e.g. a melee chaser, a ranged attacker and a patrolling guard).

Procedural trap placement: Place traps (spike pits, arrow launchers, flame jets, pressure plates, poison pools) throughout corridors and rooms using procedural rules. Trap density and type should vary based on room purpose and difficulty. Ensure traps are detectable and avoidable by a skilled player.

Procedural loot and item generation: Generate collectible items, treasure chests, health pickups and equipment using randomised loot tables. Consider weighting drop rates based on dungeon progression and difficulty. Chests should be placed in contextually appropriate locations (corners, dead ends, treasure rooms).

Dungeon furniture and props: Place contextually appropriate props in rooms — tables, barrels, bookshelves, weapon racks, crates — using rule-based or grammar-based placement systems. For example, a library room template might favour bookshelves and candles, whilst a barracks room favours weapon racks and beds.

Task 5 — Procedural Vegetation and Organic Growth [7]

Criterion assessed: A&A3

Dungeons are not sterile environments. Over time organic life creeps into abandoned spaces. In this task, you will implement procedural systems for generating vegetation and organic elements within the dungeon.

Requirements:

Procedural moss, fungi and vine growth: Implement a system that procedurally spreads moss, lichen, fungi or vines across dungeon surfaces. Use techniques such as cellular automata growth simulation, L-systems for branching vine patterns or noise-based probability maps to determine where organic growth appears. Growth should favour damp areas, corners and abandoned rooms.

Procedural tree roots and undergrowth: In certain areas of the dungeon (perhaps near surface-level rooms or collapsed ceilings), generate tree roots breaking through walls or floors, grass patches or mushroom clusters. These should be placed procedurally and should interact sensibly with the dungeon geometry.

Growth as a gameplay signal: Consider using vegetation density as a visual cue for the player. Heavily overgrown areas might indicate rooms that have been untouched for longer and could contain better loot or more dangerous enemies. This ties your procedural vegetation into the broader game design.

Task 6 — Dynamic Difficulty Adjustment [7]

Criterion assessed: A&A4

Rather than designing separate difficulty levels or multiple hand-crafted stages, your game must feature a dynamic difficulty adjustment (DDA) system that continuously adapts the dungeon's challenge to match the player's skill.

Requirements:

Player performance tracking: Implement a system that monitors player performance in real time. Track meaningful metrics such as: time taken to clear rooms, number of kills versus deaths, damage taken per encounter, health remaining after combat, accuracy of attacks and items used. Combine these metrics into an internal difficulty score or rating.

Dynamic difficulty response: Use the difficulty score to adjust the dungeon's challenge in real time or when generating the next dungeon. Adjustments should affect multiple systems across your project. For example: increase or decrease enemy count and aggression (Task 4), adjust trap density and trap types (Task 4), modify loot drop quality and frequency (Task 4) and alter vegetation overgrowth that may obscure visibility (Task 5).

Seamless adaptation: The difficulty adjustment should feel natural, not jarring. Avoid sudden jumps in difficulty. The player should perceive the game as consistently challenging without realising the system is actively adapting to them.

4. Submission Requirements

4.1 Deliverables

You must submit the following:

1. Complete Unity 6 project — submitted as a Git repository (GitHub, GitLab or Plastic SCM). The repository must contain your full project source, including all scripts, assets, scenes and your AI-generated tileset(s). Ensure the project compiles and runs without errors in Unity 6.
2. Video presentation — a screen-recorded video of no more than **15 minutes** in which you demonstrate your game and explain your procedural systems. See section 4.3 for details.

4.2 Version Control

You must use Git or Plastic SCM throughout the development of your project. Your commit history should demonstrate steady, incremental progress over the duration of the assignment. A repository with a single large commit submitted at the deadline will raise academic integrity concerns and may result in a reduced mark.

Aim for meaningful commit messages that describe what was implemented or changed in each commit. Your version control history will be reviewed as part of the marking process.

4.3 Video Presentation

Record a screen-capture video (maximum 15 minutes) in which you:

1. Demonstrate your game running — show the dungeon generating, the player navigating through it, enemies and traps in action and the difficulty adjusting over the course of play.
2. Walk through your code — open your key scripts in the editor and explain how your procedural algorithms work. Focus on the logic and decision-making behind your implementations, not on reading code line by line.
3. Explain your design decisions — discuss why you chose specific algorithms (e.g. BSP over cellular automata), what challenges you encountered and how you solved them.
4. Show your AI tileset generation process — briefly demonstrate or discuss the prompt iteration you went through to produce your tileset.

Note: You may be asked to present your work to the rest of the class and answer questions about your code and design decisions. Be prepared to explain and defend your implementation choices in a live setting.

5. Marking Rubric

The rubric below indicates the quality thresholds for each mark band. Marks are awarded based on the quality, completeness and sophistication of the work submitted for each task. Half marks may be awarded where performance falls between two descriptors.

Task 1 — AI-Generated Dungeon Tileset | Prerequisite (no separate marks) | Assessed through Tasks 2–5

Mark Band	Descriptor
High quality	A polished, cohesive AI-generated tileset that works seamlessly in Unity's Tilemap. Tiles are correctly aligned, visually consistent and cover a wide variety of dungeon elements. Evidence of thorough prompt iteration.
Acceptable	A functional tileset with minor alignment or style inconsistencies. Covers the core tile types needed for dungeon generation. Some evidence of prompt refinement.
Below expectations	A tileset with significant problems (misalignment, style clashes, limited variety). May require substantial manual correction to be usable. Little evidence of iteration.
Missing/unusable	No AI-generated tileset provided or the output is entirely unusable. This will severely limit marks across Tasks 2–5.

Task 2 — Procedural Dungeon Layout Generation | 20 marks | R&U3 (5) + R&U6 (5) + E&C3 (10)

Mark Band	Descriptor
Excellent (18–20)	The dungeon generator reliably produces varied, interesting and fully connected layouts. Multiple generation techniques are combined. Seed-based reproducibility is implemented. The student demonstrates clear understanding of the graphics pipeline in their video presentation.
Good (14–17)	The generator produces playable and varied layouts with rooms and corridors. At least one PCG terrain technique is applied effectively. The student shows reasonable understanding of the graphics pipeline. Minor issues may occasionally arise.
Satisfactory (10–13)	The generator produces basic but functional dungeon layouts. Rooms and corridors connect, but the results may lack variety. Some attempt to use PCG terrain techniques is evident. Graphics pipeline understanding is partial.
Poor (5–9)	The generator produces layouts with significant problems (disconnected rooms, broken geometry). Little or no application of PCG terrain techniques. Limited understanding of the graphics pipeline.
Inadequate (0–4)	No procedural generation is evident or the generator fails to produce usable results. No engagement with PCG terrain techniques or graphics pipeline concepts.

Task 3 — Procedural Terrain and Environmental Variation | 10 marks | E&C2 (10)

Mark Band	Descriptor
Excellent (9–10)	Rich environmental variation with noise-driven tile selection, multiple distinct biomes/zones and contextually appropriate decoration placement. Terrain has gameplay significance.
Good (7–8)	Clear environmental variation with noise-based tile changes and some zoning. Decorations are procedurally placed with reasonable context sensitivity.
Satisfactory (5–6)	Basic terrain variation is present (e.g. occasional different floor tiles) but lacks sophistication. Limited or no zoning system. Some procedural decoration placement.
Poor (3–4)	Minimal terrain variation. Floors are mostly uniform with only token differences. Little evidence of noise-based or procedural approaches.
Inadequate (0–2)	No terrain variation beyond the basic dungeon structure from Task 2.

Task 4 — Procedural Object, Trap and Enemy Spawning | 17 marks | A&A2 (7) + E&C1 (10)

Mark Band	Descriptor
Excellent (15–17)	Comprehensive procedural spawning system with multiple enemy types, varied traps, context-sensitive loot and rule-based furniture placement. All systems interact coherently.
Good (12–14)	Solid spawning systems for enemies, traps and loot with clear procedural logic. Minor gaps in variety or context sensitivity.
Satisfactory (9–11)	Basic enemy and trap spawning is procedural, but systems may be simplistic. Limited loot variation.
Poor (5–8)	Only one or two object types are spawned procedurally. Little variety in enemies or traps.
Inadequate (0–4)	Objects are placed manually or spawning is non-functional. No procedural logic for game objects.

Task 5 — Procedural Vegetation and Organic Growth | 7 marks | A&A3 (7)

Mark Band	Descriptor
Excellent (6–7)	Sophisticated organic growth system using cellular automata, L-systems or similar. Vegetation responds to dungeon context (damp areas, corners). Growth serves as a meaningful gameplay signal.
Good (5)	Functional vegetation system with procedural placement. Growth patterns are context-aware and visually convincing, though may lack complexity.
Satisfactory (3–4)	Basic vegetation is placed procedurally but without strong context sensitivity. Growth patterns are simple but present.
Poor (1–2)	Minimal vegetation with little or no procedural logic. Plants appear randomly with no consideration of context.
Inadequate (0)	No procedural vegetation is implemented.

Task 6 — Dynamic Difficulty Adjustment | 7 marks | A&A4 (7)

Mark Band	Descriptor
Excellent (6–7)	A well-tuned DDA system that tracks multiple player metrics and adjusts several game parameters smoothly. Adaptation feels seamless.
Good (5)	A functional DDA system tracking key metrics with visible adjustments to game difficulty.
Satisfactory (3–4)	A basic DDA system that tracks at least one metric and adjusts at least one game parameter.
Poor (1–2)	A rudimentary difficulty system with minimal tracking and limited impact on gameplay.
Inadequate (0)	No DDA system is implemented.

6. Academic Integrity

This is an individual assignment. All code, written work and design decisions must be your own. You may use the following resources:

- Unity documentation and official tutorials
- Publicly available PCG algorithm references and academic papers
- Generative AI tools for tileset creation (Task 1) — clearly document your prompts and iteration process
- AI coding assistants (e.g. GitHub Copilot, ChatGPT) for help with specific coding problems — you must understand and be able to explain every line of code in your submission

You must not submit work produced by another person, copy code from other students or submit substantial AI-generated code that you cannot explain. If you are asked to present your work and cannot adequately explain your implementation, your mark may be reduced or the case may be referred under the university's academic misconduct procedures.

Good luck and enjoy building your dungeon!